GRID DP — the bridge between DP and graphs

```
// dp[r][c] depends on dp[r-1][c] and/or dp[r][c-1] (values from above/left)
// Same 'skip or take' DP mindset from Page 18, applied to 2D movement instead
// of a 1D array — only right/down moves means no cycles, so DP applies cleanly.
```

DEMO 1: MIN PATH SUM (top-left to bottom-right, right/down only)

Use when: minimizing/maximizing a cost while moving only right or down.

```
vector<vector<long long>> dp(R, vector<long long>(C, 0));
dp[0][0] = grid[0][0];
for (int c=1;c<C;c++) dp[0][c] = dp[0][c-1] + grid[0][c];  // first row: only from
left
for (int r=1;r<R;r++) dp[r][0] = dp[r-1][0] + grid[r][0];  // first col: only from
above
for (int r=1;r<R;r++)
    for (int c=1;c<C;c++)
        dp[r][c] = grid[r][c] + min(dp[r-1][c], dp[r][c-1]);
// answer = dp[R-1][C-1]
```

DEMO 2: COUNT UNIQUE PATHS (right/down only, no costs)

```
vector<vector<long long>> dp(R, vector<long long>(C, 0));
for (int r=0;r<R;r++) dp[r][0] = 1;  // only 1 way along first column
for (int c=0;c<C;c++) dp[0][c] = 1;  // only 1 way along first row
for (int r=1;r<R;r++)
    for (int c=1;c<C;c++)
        dp[r][c] = dp[r-1][c] + dp[r][c-1];  // sum of ways, not min
// answer = dp[R-1][C-1]
```

DEMO 3: UNIQUE PATHS WITH OBSTACLES

Same as Demo 2, but blocked cells contribute 0 ways.

```
vector<vector<long long>> dp(R, vector<long long>(C, 0));
dp[0][0] = (grid[0][0] == 1) ? 0 : 1;  // 1 = obstacle
for (int r=0;r<R;r++)
    for (int c=0;c<C;c++) {
        if (grid[r][c] == 1) { dp[r][c] = 0; continue; }
        if (r>0) dp[r][c] += dp[r-1][c];
        if (c>0) dp[r][c] += dp[r][c-1];
    }
```

SPACE OPTIMIZATION NOTE

```
// if dp[r][c] only needs row r-1, keep just ONE row array and overwrite in place
// (same rolling-variable idea as Page 18, extended to a 1D row instead of scalars)
```

===== READ GRID OF NUMBERS INTO 2D VECTOR =====

```
vector<vector<long long>> grid(R, vector<long long>(C));
for (int i=0;i<R;i++)
    for (int j=0;j<C;j++)
        cin >> grid[i][j];
```

===== FORMULA: number of monotonic paths (no obstacles), combinatorial =====

```
// paths from (0,0) to (R-1,C-1) moving only right/down = C(R+C-2, R-1)
// useful for verifying your DP answer, or when R,C large and no obstacles exist
// ===== COMMON GRID DP TRANSITION =====
// dp[r][c] usually combines values from top and left
// first row/column usually initialized separately
```

8 Directions

```
int dx8[] = {-1,-1,-1,0,0,1,1,1};
int dy8[] = {-1,0,1,-1,1,-1,0,1};
```